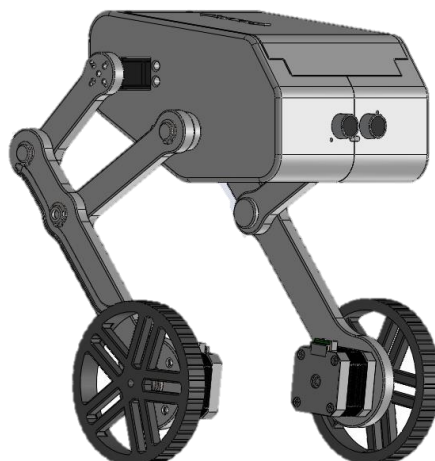




Wheel-Legged Robot

MET 331: Automatic Control 1



Team Members

Amr Mohamed

Abdelrahman Hatem

Mina Sameh

Doaa Essam

Esraa Mohamed

Alyaa Mohamed

Supervisor

Assoc. Prof. Dr. Shuaiby Mohamed

Mechatronics Engineering Department
Faculty of Engineering
Assiut University

2025-2026

1 Table of Contents

1	Table of Contents	II
2	Table of Symbols	V
3	Introduction	1
3.1	Evolution of Mobile Robotics The Wheel-Legged Paradigm	1
3.2	Project Scope and Academic Context.....	1
4	Problem Statement.....	1
5	Project Objectives	1
6	Mechatronic System Design.....	2
6.1	Mathematical Modeling for the system	2
6.2	Control System Configuration	2
6.3	Physical system design (Mechanical)	2
6.4	Embedded Hardware in Robot	2
6.5	Mechanical Design.....	2
6.6	Electronic Hardware and Sensor Integration	4
6.6.1	Central Processing Unit (ESP32).....	4
6.6.2	Attitude Estimation (MPU6050 IMU & Filtering)	4
6.6.3	Ultrasonic	5
6.6.4	Actuation System (Stepper Motors & Servos)	5
6.6.5	Power Management (LiPo Battery Weight Reduction).....	5
7	Mathematical Modelling.....	6
7.1	Dual-Wheel Inverted Pendulum Model.....	6
7.1.1	The equations of Inverted Pendulum.....	6
7.1.2	Four-Link Motion Model	8
8	Control Strategy	10
8.1	PD control	10
8.1.1	Error Formulation and Controller Optimization [1]	10
8.1.2	Gravity Feedforward Compensation [1].....	10

8.2	Virtual Model Control (VMC) and Joint Torque Mapping [1]	10
8.2.1	Jacobian Formulation and Force Translation [1]	11
8.2.2	PD control & Four bar linkage Model in Simulink	11
8.3	LQR with Dynamic Modeling & State-Space Expansion	12
8.3.1	Baseline 4-State Longitudinal Model $\mathbf{x}, \dot{\mathbf{x}}, \boldsymbol{\theta}, \dot{\boldsymbol{\theta}}$	12
8.3.2	Advanced 6-State Spatial Model (<i>Integration of Yaw $\delta, \dot{\delta}$</i>)	13
8.4	Look-up Polynomial Fitting & LQR Gain Scheduling	14
8.4.1	Offline MATLAB Curve Fitting for Kinematics	14
8.4.2	Real-Time LQR Matrix Adjustment	14
9	Simulation and Experimental Results	15
9.1	Simscape Multi-Body Simulation Environment	15
9.1.1	CAD Integration and Virtual Physics Setup	15
9.2	Control Verification	16
	Static Pitch Stabilization (Constant θ_1)	16
10	Discussion	18
10.1	Computational Efficiency of Polynomial Gain Scheduling	18
10.2	Multi-Environment Simulation and Control Verification	18
10.3	Limitations and Future Structural Improvements	18
11	Conclusion	19
12	References	20
13	Appendices	21
13.1	Appendix A: Low-Level Hardware Diagnostics and Unit Testing Scripts	21
13.2	Appendix B: Embedded Control Firmware (C++/Arduino Platform)	23
13.3	Appendix C: Dynamic LQR State-Space Controller Implementations	27
13.4	Appendix D: High-Level Simulation and Control Initialization Scripts	29
13.5	Appendix E: Offline MATLAB Numerical Fitting and Matrix Scheduling Scripts	31

Table of Figures

Figure 1 linkage fixdiation	2
Figure 2 Side View	2
Figure 3 side view of linkage	3
Figure 4 3d printed part	3
Figure 5 Assembled robot	3
Figure 6 Exploded view	3
Figure 7 Testing stepper motor.....	4
Figure 8 MPU wiring.....	4
Figure 9 stepper driver wiring.....	5
Figure 10 modeling systems	6
Figure 11 Inverted Pendulum Model	6
Figure12 Inverted pendulum model in Simulink	7
Figure 13 Four-Link Motion Model	8
Figure 14 PD Response.....	11
Figure 15 PD response with ramp mixed step input.....	12
Figure 16: LQR with 4 states	13
Figure 17 : LQR with 6 states	13
Figure 18 validation k matrix gains	14
Figure 19 simscape model	15
Figure 20 simulink model with simscape multibody.....	16
Figure 21 pitch angle vs Ref Pitch with time	17
Figure 22 simscape balaning on 0 rad.....	17

2 Table of Symbols

Table 1. Parameter definitions of inverted pendulum model.

Symbol	Meaning	Unit
M	Body mass	kg
m	Wheel mass	kg
l	Distance from body center of mass to wheel center	m
r	Wheel radius	m
D	Distance between left and right wheels	m
g	Gravitational acceleration	m/s ²
$\theta, \dot{\theta}, \ddot{\theta}$	Machine pitch angle, angular velocity, angular acceleration	rad, rad/s, rad/s ²
T _L , T _R	Driving torque of the wheel	N·m
N _L , N _R , P _L , P _R	Force between the body and the wheel in the x and y directions	N
F _L , F _R	Friction between the wheel and the ground	N
x _L , x _R , $\dot{x}_L$, $\dot{x}_R$, $\ddot{x}_L$, $\ddot{x}_R$	Wheel displacement, velocity, acceleration	m, m/s, m/s ²
ω_L , ω_R , $\dot{\omega}_L$, $\dot{\omega}_R$	Wheel rotation angular velocity and acceleration	rad/s, rad/s ²
I	Wheel moment of inertia	kg·m ²
J _x , J _y , J _z	Moment of inertia of the body around the x, y, z axes	kg·m ²
$\delta, \dot{\delta}, \ddot{\delta}$	Course angle, angular velocity, angular acceleration	rad, rad/s, rad/s ²

Table 2. Parameter definitions of the wheel-legged robot model.

Symbol	Meaning	Unit
l ₁ , l ₂ , l ₃ , l ₂₃	Length of the corresponding rod	m
l ₄	Distance between the joints	m
l ₅	Distance from the intersection point of the upper joint to the lower lever	m
l _{leg}	Pendulum height	m
θ_1	Angle between bar l ₂ and the horizontal axis	rad
θ_2	Angle between auxiliary connecting rod l ₄ and l ₅	rad
θ_3	Angle between auxiliary connecting rod l ₅ and l ₂₃	rad
θ_4	Angle between bar l ₁ and the horizontal axis	rad
θ_5	Angle between bar l ₂₃ and the horizontal axis	rad
θ_6	Angle between bar l ₃ and the horizontal axis	rad
L _{desired}	Desired leg length	m
x _{c2} , x _{c1} , x _{c3} , x _{c23}	Vertical coordinates of the center of mass of the connecting rod	m
L ₀	Height of the total center of gravity of the system	m
l _w	Height of the center of gravity of the pendulum	m
R	Drive wheel radius	m
m _w	Drive wheel mass	kg
m _b	Body mass	kg
m _i	Pendulum mass	kg
m _p	Total system mass	kg
L ₁	Distance from center of gravity to wheel axle	m
I _{w1}	Drive wheel moment of inertia	kg·m ²
I _{p1}	Moment of inertia of a pendulum	kg·m ²
N	Normal force	N
P	Combined force of gravity and acceleration	N
T	Driving torque	N·m

3 Introduction

3.1 Evolution of Mobile Robotics The Wheel-Legged Paradigm

For intelligent mobile robots, the Wheel-Legged Robot is dealt with as high maneuverability, easy transformation, and efficient animal-based design robots. The evolution of these platforms seeks to bridge the gap between the speed of traditional wheeled rovers and the terrain adaptability of legged variants.

3.2 Project Scope and Academic Context

This project enabled us in integrating variable subsystems in one project being such impressive Mechatronic system enabling. From Mathematical equations, Mechanical design, Embedded system integration, and Control system design, the scope covers the full lifecycle of mechatronic product development.

4 Problem Statement

Developing a unified wheel-legged robot is dealt with as a complex mechatronic struggle where the gap between theoretical mathematical models and physical embedded hardware often results in poor system performance. The non-linear multi-body dynamics inherent in combining a four-bar linkage with an inverted pendulum create significant computational bottlenecks for standard embedded controllers.

This project enabled us in overcoming the difficulty of synchronizing mechanical link dynamics with real-time control algorithms being a fully integrated mechatronic achievement. From mechanical design complexities, mathematical modeling errors, embedded integration hurdles, and control system instability.

5 Project Objectives

For achieving high maneuverability and stability, this project aims in developing advanced hybrid control systems. This requires structural optimization of the linkage mechanisms, mathematical optimization through polynomial fitting to handle dynamic shifts, and scaling the control architecture to manage varied actuators.

This project enabled us in creating accurate mathematical models being such a precise mechatronic integration. From LQR for balance, PD for height control, and Fitting Techniques for real-time CoM prediction with considering applying these strategies in a real-world plant.

6 Mechatronic System Design

6.1 Mathematical Modeling for the system

Treating physical plant as two subsystems: height subsystem or four bar linkage to model the variation in height and leg mechanism, and inverted pendulum subsystem to model the balancing function of the robot and its motion.

6.2 Control System Configuration

Based on mathematical model made and out understanding of the physical functions needed, we made the control strategy to be in two systems: PD control to manage height, LQR control to manage balance and motion.

6.3 Physical system design (Mechanical)

Here we created the physical plant to be established in real-world. The plant design must fit configurations made in mathematical model from masses and dimensions.

6.4 Embedded Hardware in Robot

The physical plant to be actuated we need actuators to do the functions needed, sensors to close the control loop, and controller itself to manage all that.

6.5 Mechanical Design

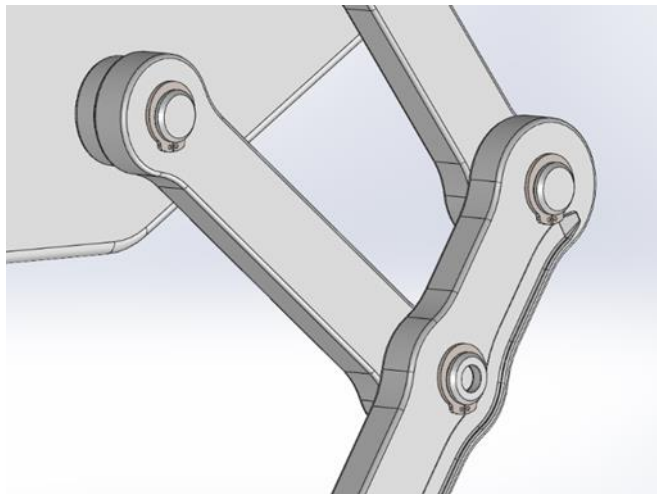


Figure 1 linkage fixdiation

The design is made mainly to be easy in manufacturing and assembly. From selecting 3D printed PLA to be the material used, we customized the design on it. The design is easy in assembly by using snap-rings, interference fit assembly, and easy pin usage with respect of suitable dimensions.

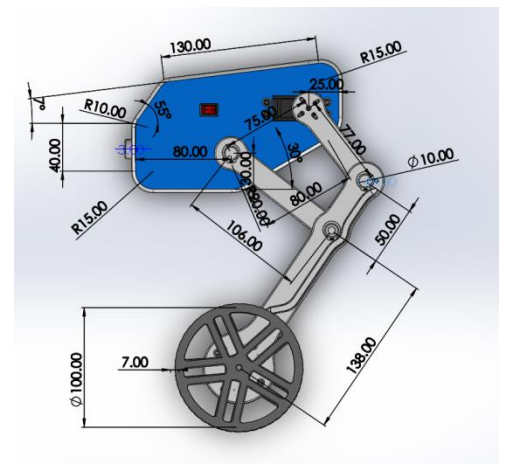


Figure 2 Side View

Also for wire management, we made a slot in stepper-motor's link to act like a duct for wires to manage in.



Figure 4 3d printed part

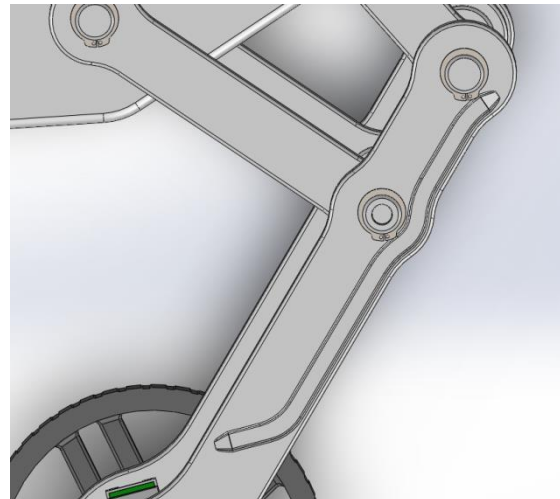


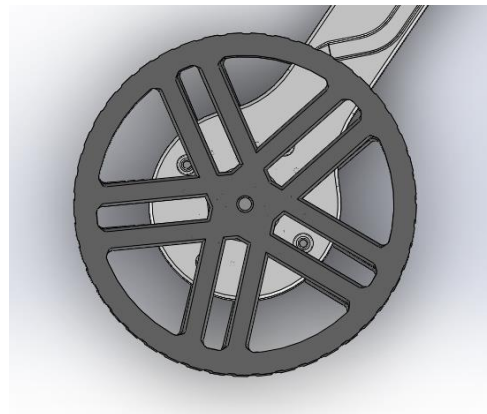
Figure 3 side view of linkage

In wheel, it's critical area where we need to decrease friction between wheel and link, and increase friction in suitable area between wheel and ground to keep balance and stable and move right.

We designed the wheel to have a good friction surface with ground, ribs are made to avoid stresses on hip and make two direction of reaction torques from ground to wheel what make a passive balance.



Figure 5 Assembled robot



Also considered bearing assembly between wheel and motor's shaft to keep smooth motion.

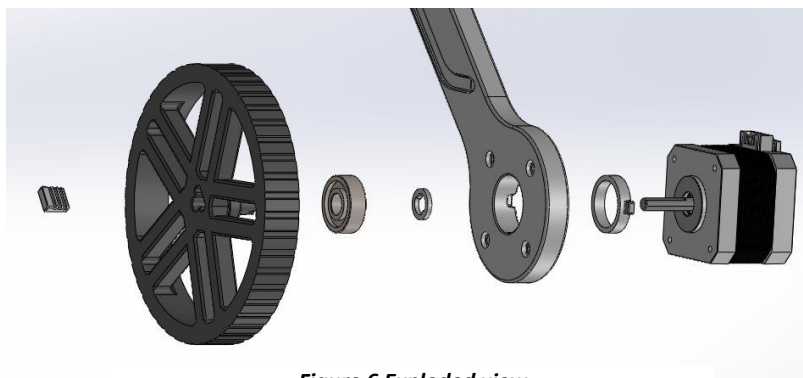


Figure 6 Exploded view

6.6 Electronic Hardware and Sensor Integration

The robot hardware consists of sensing, control, and actuation components integrated together to achieve self-balancing operation.

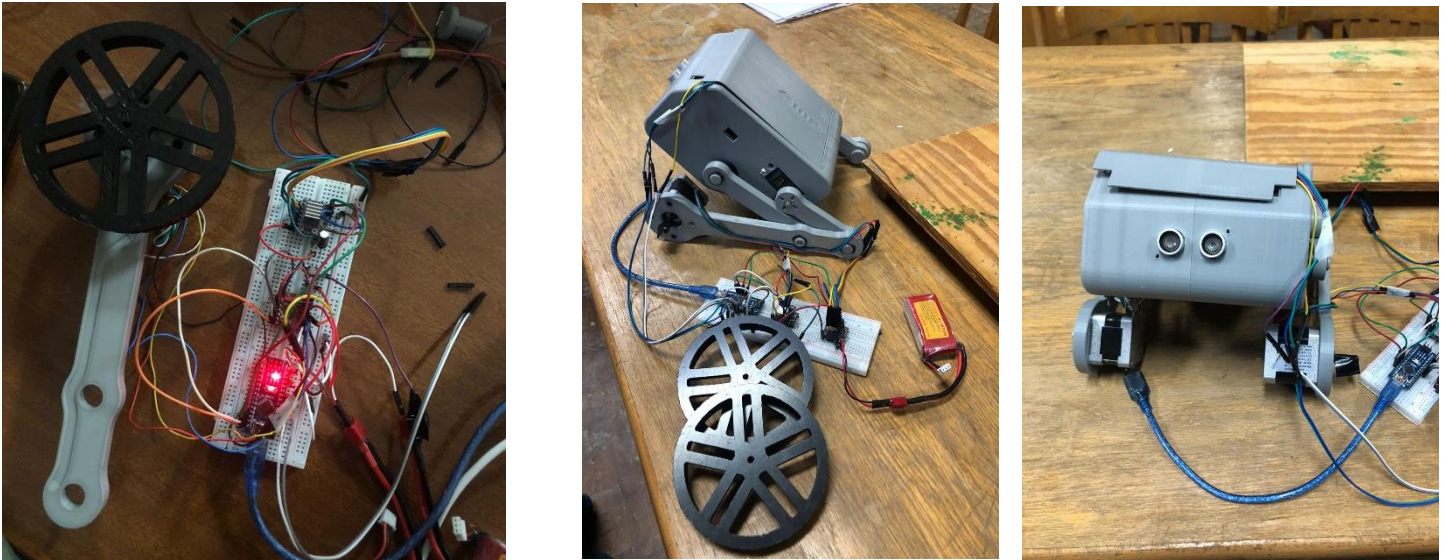


Figure 7 Testing stepper motor

6.6.1 Central Processing Unit (ESP32)

The main controller used in the system is the Arduino Nano, which is responsible for processing sensor data and generating control signals for the motors.

An ESP32 module was also considered for future development to provide wireless communication, remote monitoring, and real-time data transmission capabilities.



6.6.2 Attitude Estimation (MPU6050 IMU & Filtering)

The robot uses the MPU6050 inertial measurement unit to measure the robot orientation and angular velocity.

The sensor communicates with the Arduino Nano using the I2C protocol through the SDA and SCL pins.

The MPU6050 sensor is connected to the Arduino Nano using the I2C communication protocol. The SDA pin of the MPU6050 is connected to pin A4 of the Arduino Nano, while the SCL pin is connected to pin A5. The sensor is powered using the 5V and GND pins of the Arduino. Through this connection,

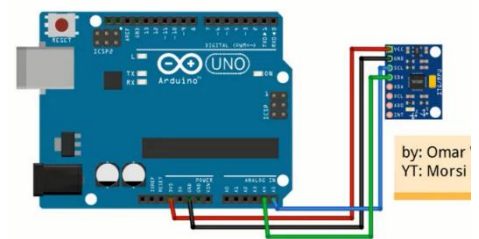


Figure 8 MPU wiring

the microcontroller continuously receives accelerometer and gyroscope measurements required for estimating the robot orientation and angular velocity

6.6.3 Ultrasonic

Additionally, an ultrasonic sensor was prepared for future implementation in obstacle detection and environmental interaction. The sensor has not yet been integrated into the software because the current stage of the project focuses mainly on achieving stable balancing performance.

6.6.4 Actuation System (Stepper Motors & Servos)

6.6.4.1 Stepper motors

Two stepper motors are used as the main actuation system for balancing and movement. Each motor is driven using a DRV8825 Stepper Motor Driver module, which receives STEP and DIR signals from the microcontroller to control motor speed and rotation direction with high precision.

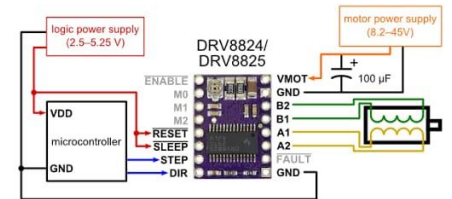


Figure 9 stepper driver wiring

6.6.4.2 Servo motors

The system also includes two servo motors intended for adjusting the robot leg configuration and varying the effective pendulum length during operation. However, in the current implementation, the leg length is assumed to be fixed; therefore, the servo motors are not yet integrated into the code in arduino .

6.6.5 Power Management (LiPo Battery Weight Reduction)

A high-discharge Lithium-Polymer (LiPo) battery pack was selected to power the system instead of traditional heavy batteries, due to its superior energy density and lightweight profile. This choice minimized the global weight of the prototype and successfully delivered the high current spikes required by the NEMA 17 stepper motors during rapid balancing corrections, while placing it at the bottom of the chassis base successfully lowered the center of mass to enhance structural stability.

7 Mathematical Modelling

The wheeled bipedal robot is a complex nonlinear multiple-input multiple-output system, which regards the robot as a combination of left and right legs, left and right wheels, plus the body, and left and right symmetry, so an idealized setting can be made.

The robotic dynamic model is decomposed into wheel motion and leg motion. The wheel subsystem, responsible for linear locomotion and steering, is simplified as a two-wheel inverted pendulum model. The leg subsystem employs a four-bar linkage mechanism to achieve jumping, full-body height adjustment, and adaptive locomotion on uneven terrain. By focusing on analyzing the dynamic model of the four-bar linkage motion, the originally coupled complex model is streamlined. As shown in Figure 1, this decoupling strategy simplifies the design process while retaining critical functionalities, thus enabling efficient control of the robot's hybrid mobility modes.

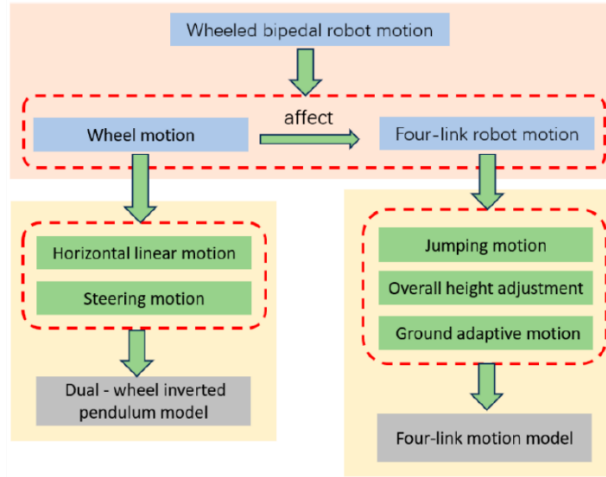


Figure 10 modeling systems

7.1 Dual-Wheel Inverted Pendulum Model

The mass of the body is equivalently concentrated in the position of the center of mass. Ignoring the effect of the leg motion on the wheeled movement and the mass of the four links, the driving wheels are in non-slip rolling friction with the ground. Therefore, the four-link wheel-legged robot can be equivalently simplified into a two-wheeled inverted pendulum model with variable rod length, see Figure 2 and Table 1.

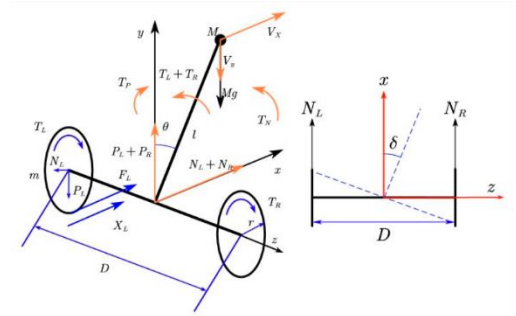


Figure 11 Inverted Pendulum Model

7.1.1 The equations of Inverted Pendulum

7.1.1.1 Planar Linearization and Baseline Dynamics

To design a linear optimal controller such as the Linear Quadratic Regulator (LQR), the highly non-linear equations of motion derived via the Euler-Lagrange framework must be linearized around the upright equilibrium point. When the pitch inclination of the main body (θ) is small ($\theta \approx 0$), the small-angle approximation can be applied safely. Under these assumptions, higher-order differential terms are neglected, and the trigonometric variables are linearized as follows

$$\cos \theta \approx 1, \quad \sin \theta \approx \theta, \quad \dot{\theta}^2 \approx 0$$

Applying these conditions to the continuous planar dynamics yields the coupled, linearized differential equations governing longitudinal displacement (x) and body pitch (θ) [1]:

$$\left(M + 2m + \frac{2I_w}{r^2}\right)\ddot{x} = \frac{T_L + T_R}{r} - Ml\ddot{\theta}$$

$$(J_z + Ml^2)\ddot{\theta} = Mgl\theta - Ml\ddot{x} - (T_L + T_R)$$

These coupled relations serve as the direct mathematical basis for the 4-state state-space block

$$(X_4 = [x, \dot{x}, \theta, \dot{\theta}]^T)$$

constructed within the MathWorks Simulink modeling workspace to evaluate initial straight-line balancing performance.

7.1.1.2 Steering Kinematics and Yaw Modeling

To achieve full 3D planar mobility, differential steering capabilities must be integrated alongside the longitudinal balancing routines. By sorting the mechanical equations of constraint across the transverse horizontal frame, the uncoupled dynamic equation governing the steering yaw angle (δ) is formulated as follows [1]:

$$\ddot{\delta} = \frac{T_L - T_R}{r \left(mD + \frac{I_D}{r^2} + \frac{2J_y}{D} \right)}$$

This yaw relation is mathematically superimposed over the planar balancing matrix to build out the full 6-state spatial LQR vector ($X_6 = [x, \dot{x}, \theta, \dot{\theta}, \delta, \dot{\delta}]^T$). Within Simulink, this decoupled torque relationship ($\Delta T = T_L - T_R$) is utilized by the controller to actively adjust steering headings without injecting unstable oscillations into the main pitch axis (θ).

Inverted pendulum model in Simulink

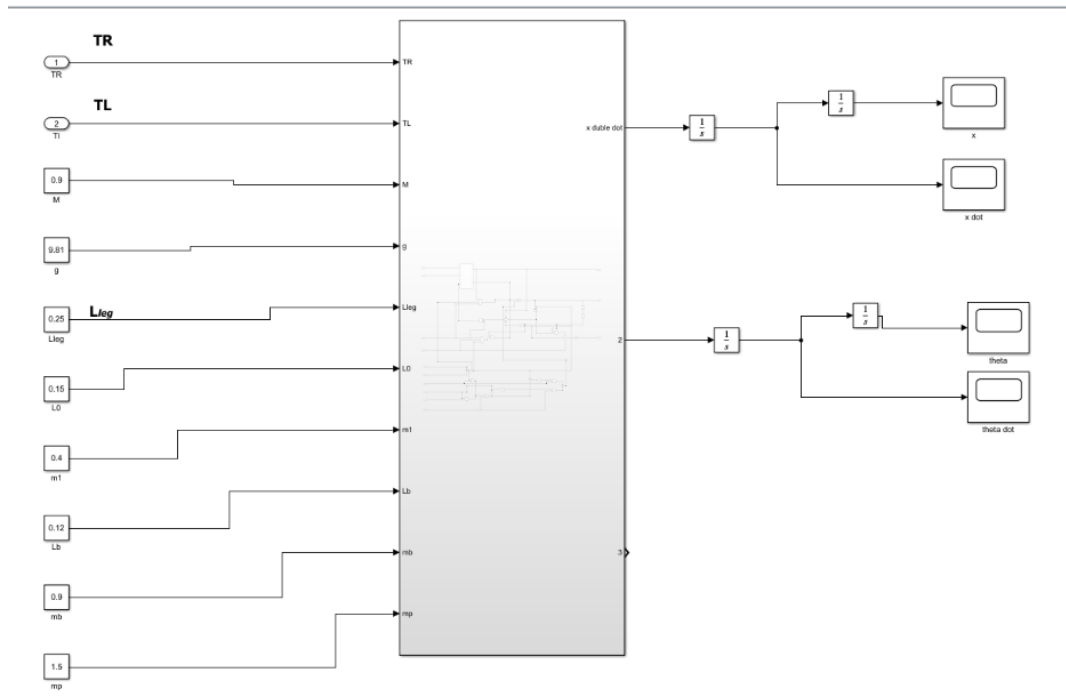


Figure12 Inverted pendulum model in Simulink

7.1.2 Four-Link Motion Model

The whole machine is symmetrical on the left and right, the kinematics of the four-link structure of the left leg are analyzed in detail see figure , and the relevant parameters are illustrated in **Table 2** .

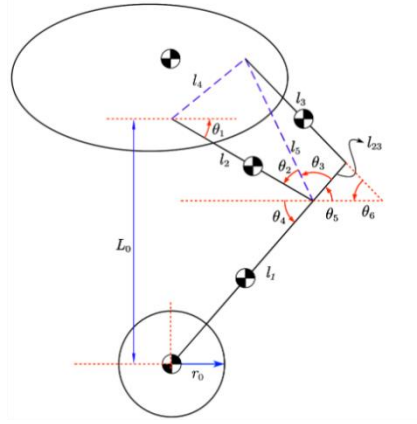


Figure 12 Four-Link Motion Model

7.1.2.1 The equations of Four-Link Mechanism

Given the instantaneous command angle of the joint servo motor θ_1 , the secondary linkage parameters ($\theta_2, \theta_3, \theta_4$) and the final effective leg length (l_{leg}) are computed explicitly as follows:

$$l_5 = \sqrt{l_2^2 + l_4^2 - 2 \cdot l_2 \cdot l_4 \cdot \cos\left(\theta_1 + \frac{\pi}{4}\right)}$$

$$\theta_2 = \arcsin\left(\frac{l_4 \cdot \sin\left(\theta_1 + \frac{\pi}{4}\right)}{l_5}\right)$$

$$\theta_3 = \arccos\left(\frac{l_{23}^2 + l_5^2 - l_3^2}{2 \cdot l_{23} \cdot l_5}\right)$$

$$\theta_4 = \pi - \theta_1 - \theta_2 - \theta_{leg} = l_2 \cdot \sin(\theta_1) + l_1 \cdot \sin(\theta_4)$$

7.1.2.2 Inverse Kinematics (Leg Length to Angle)

When managing height tracking commands, the target leg length (l_{leg}) is transformed back into the required actuator servo angle (θ_1) along with companion configurations (θ_5, θ_6) via the following inverted expressions:

$$\theta_1 = \arcsin\left(\frac{l_{leg}^2 + l_2^2 - l_1^2}{2 \cdot l_{leg} \cdot l_2}\right)$$

$$\theta_5 = \pi - \theta_4$$

$$\theta_6 = \arcsin\left(\frac{l_5 \cdot \sin(\theta_3)}{l_3}\right) - \theta_4$$

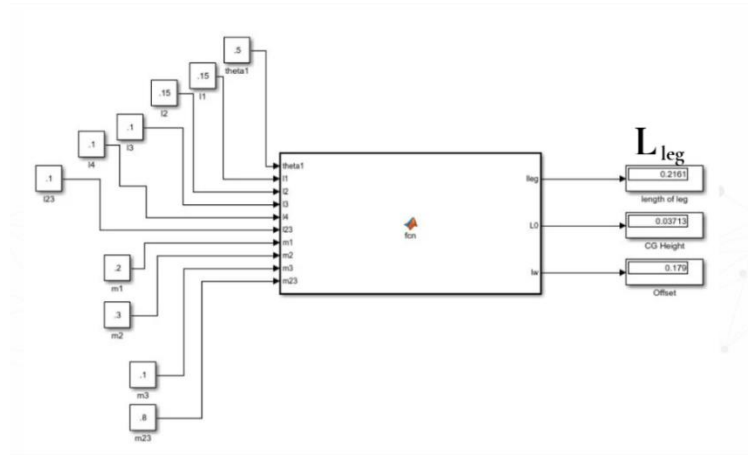
7.1.2.3 Center of Gravity Offset (l_w)

The overall vertical center of mass displacement (l_w) of the mechanical assembly relative to the wheel rotational hub is obtained directly by subtracting the linkage mass weighted average (L_0) from the total computed leg height:

$$l_w = l_{leg} - L_0$$

Where L_0 represents the composite geometric center of gravity distribution across the linkage components (m_1, m_2, m_3, m_{23}):

$$L_0 = \frac{m_1 \cdot x_{c1} + m_2 \cdot x_{c2} + m_3 \cdot x_{c3} + m_{23} \cdot x_{c23}}{m_1 + m_2 + m_3 + m_{23}}$$



8 Control Strategy

8.1 PD control

PD control is used to calculate the external force F_0 that needs to be applied on the joint motor. Omitting the I term can avoid the situation of integral saturation overshooting and improve the dynamic response speed, which is suitable for highly dynamic scenarios. The integral term will accumulate noise or high-frequency interference, which will lead to jitter of the control signal, while the differential term has a natural suppression effect on high-frequency noise, and PD control is more anti-interference.

8.1.1 Error Formulation and Controller Optimization [1]

The height controller monitors the instantaneous offset between the calculated leg length (l_{leg}) and the target setpoint ($L_{desired}$). The geometric tracking error (e) is defined as [1]:

$$e = l_{leg} + L_{desired}$$

Using this error signal, the restorative control effort (τ_{PID}) is processed via a PD loop to drive the vertical tracking error to zero :

$$\tau_{PID} = K_p e + K_d \frac{de}{dt}$$

8.1.2 Gravity Feedforward Compensation [1]

The mass of the physical robot assembly serves as a continuous constant load on the articulated joint actuators. The vertical ground support force required to hold the chassis is proportional to the total gravitational force (mg). However, due to the articulation angle of the four-bar linkages, the actual torque seen at the link joint is modulated by its geometric position. The localized joint-angle effect is captured by introducing a continuous gravity compensation component ($\tau_{gravity}$):

$$\tau_{gravity} = mg \cos(\theta_1)$$

Where $\cos(\theta_1)$ reflects the instantaneous rotation angle of the joint relative to the gravitational vector. Combining the dynamic PD response alongside this static feedforward torque yields the final required external joint force command F_0

$$F_0 = \tau_{gravity} + \tau_{PID}$$

8.2 Virtual Model Control (VMC) and Joint Torque Mapping [1]

To implement the calculated spatial force commands (F_0) using physical hardware, Virtual Model Control (VMC) is utilized. VMC creates a mathematical bridge between physical multi-body mechanics and actuator joint spaces. This architecture assumes an ideal "virtual model" (such as a structural spring-damper suspension) exists between the main chassis frame and the driving wheel hub,

8.2.1 Jacobian Formulation and Force Translation [1]

Translating the operational force F_0 into a concrete joint motor torque T_{joint} requires evaluating the geometric constraints of the closed four-bar linkage mechanism. By transforming the link angles and tracking segment lengths, the continuous system mapping is resolved using a localized scalar element of the system Jacobian matrix (J) [1]:

$$J = l_2 \cos(\theta_1) + l_1 \cos(\theta_1 + \alpha + \beta)$$

The computed external force (F_0) acting from the end-effector endpoint serves as the fundamental input variable for the virtual model. By applying the principle of virtual work, the geometric parameters and kinematic configurations are mapped cleanly to the physical actuator via the Jacobian scalar :

$$T_{joint} = F_0 J$$

This VMC framework operates in parallel with the high-speed wheel LQR tracking loops. It allows the ESP32 microcontroller to execute smooth, low-latency crouching, standing, and active landing maneuvers on the physical hardware without injecting disruptive force disturbances into the primary self-balancing control loops.

8.2.2 PD control & Four bar linkage Model in Simulink

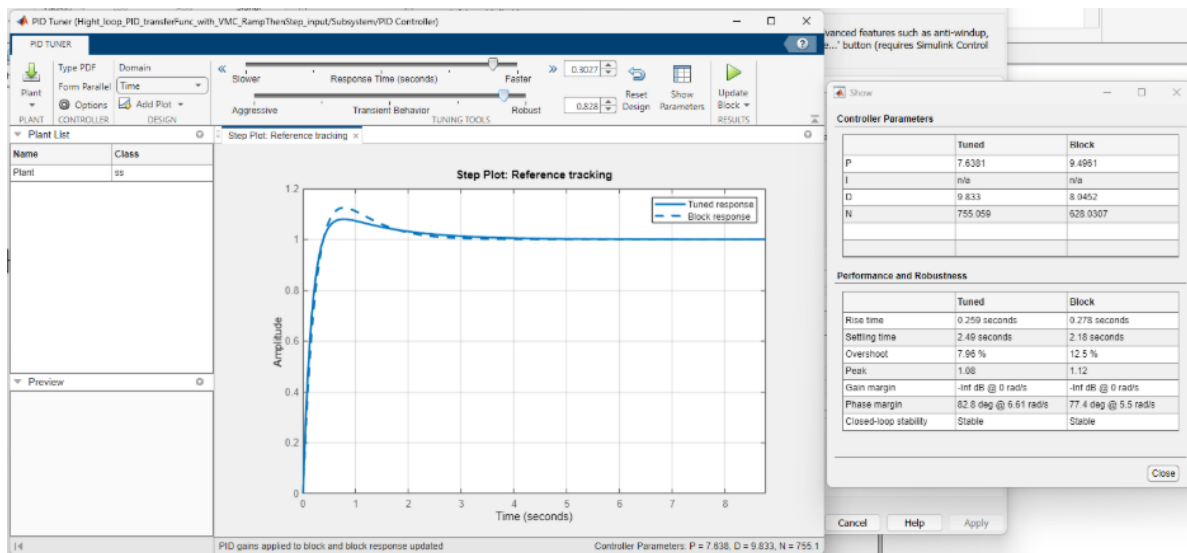
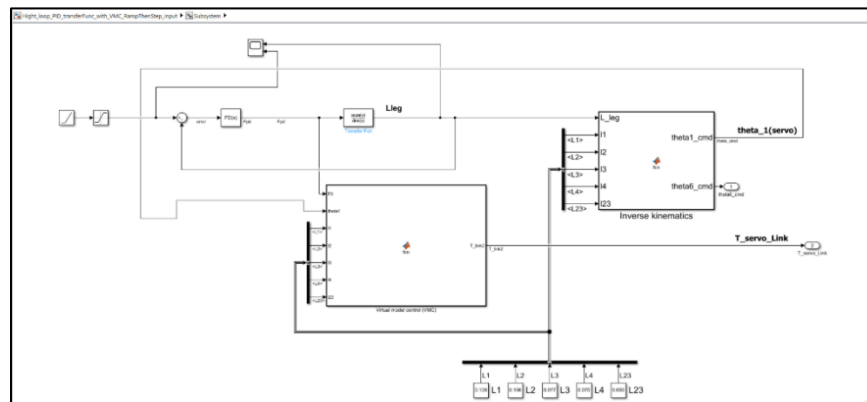


Figure 13 PD Response

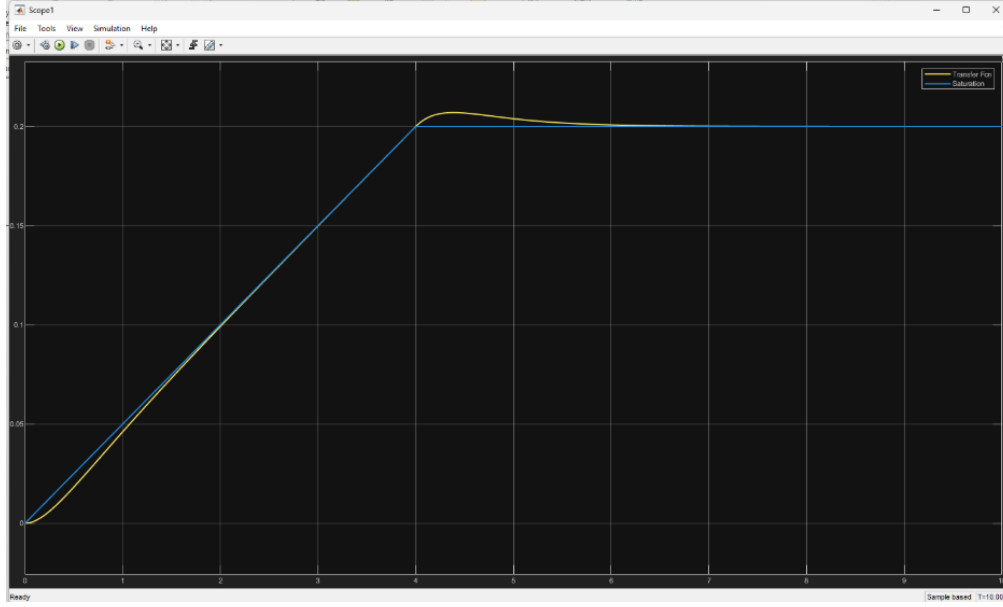


Figure 14 PD response with ramp mixed step input

8.3 LQR with Dynamic Modeling & State-Space Expansion

Due to the nonlinear and strongly coupled characteristics of wheeled bipedal robots, developing an accurate mathematical model is critical for effective control. The fundamental challenge of the TWSBR is that its center of mass (COM) and rotational inertia change continuously as the leg linkages flex and extend. To achieve robust balancing and trajectory tracking, the dynamic equations were derived using the Euler-Lagrange formalism and subsequently linearized around the upright equilibrium point to form the state-space representation

$$\dot{X} = AX + Bu$$

Recognizing the need for both longitudinal stability and spatial maneuverability, the control architecture was incrementally expanded from a planar 4-state model to a comprehensive 6-state model and tested inside MathWorks Simulink.

8.3.1 Baseline 4-State Longitudinal Model $(x, \dot{x}, \theta, \dot{\theta})$

The initial control framework treated the robot as a variable-length two-wheeled inverted pendulum restricted to purely longitudinal (forward and backward) motion. The system dynamics were captured using a 4-dimensional State Vector :

$$X_4 = [x \quad \dot{x} \quad \theta \quad \dot{\theta}]^T$$

where x and $\dot{x}$ represent the longitudinal displacement and velocity of the wheel axle, while θ and $\dot{\theta}$ denote the pitch angle and pitch angular velocity of the main body, respectively. In this planar configuration, the control input u was defined as the total symmetric torque applied to both wheels, $u = T_L + T_R$

The Linear Quadratic Regulator (LQR) was designed to minimize the cost function $J = \int_0^\infty (X^T Q X + u^T R u) dt$, where the state weight matrix Q heavily penalized deviations in the pitch angle to prevent tipping, and the control weight matrix R restricted excessive torque demands that could saturate the stepper motors.

This configuration was modeled in Simulink utilizing the native State-Space block to evaluate baseline stabilization and straight-line displacement tracking.

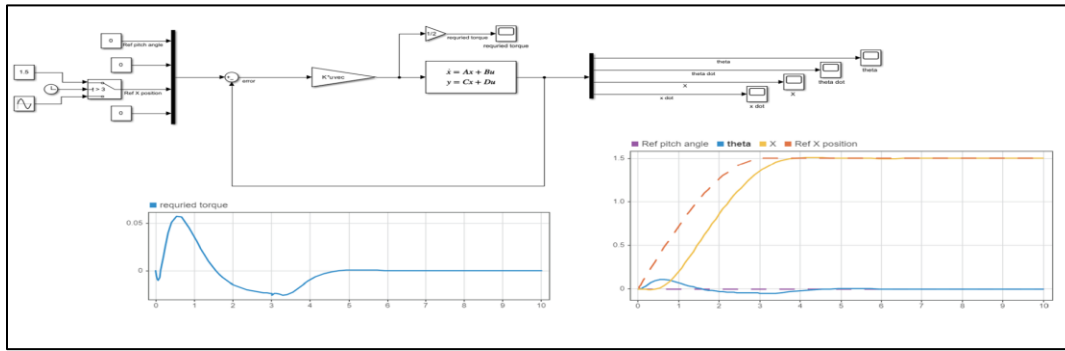


Figure 15: LQR with 4

8.3.2 Advanced 6-State Spatial Model (*Integration of Yaw $\delta, \dot{\delta}$*)

While the 4-state model successfully stabilized the robot against gravity, real-world navigation requires active steering.

The model was therefore expanded into a 6-state spatial vector to incorporate differential drive kinematics:

$$X_6 = [x \quad \dot{x} \quad \theta \quad \dot{\theta} \quad \delta \quad \dot{\delta}]^T$$

Here, δ and $\dot{\delta}$ represent the yaw (heading) angle and yaw rate. To control this expanded system, the input vector was separated into independent left and right wheel torques:

$$u = \begin{bmatrix} T_L \\ T_R \end{bmatrix}$$

This decoupling allows the LQR controller to actively command asymmetric torque ($T_L \neq T_R$) to induce a turning moment around the Z-axis, while simultaneously maintaining the symmetric torque required to keep the inverted pendulum upright.

This advanced configuration was fully implemented inside a 6-state Simulink block diagram. Virtual reference-tracking verification was conducted by feeding targeted coordinate trajectories into the controller. The resulting response curves verified that the 6-state system could successfully execute full planar trajectory following and waypoint target capture without destabilizing the pitch axis.

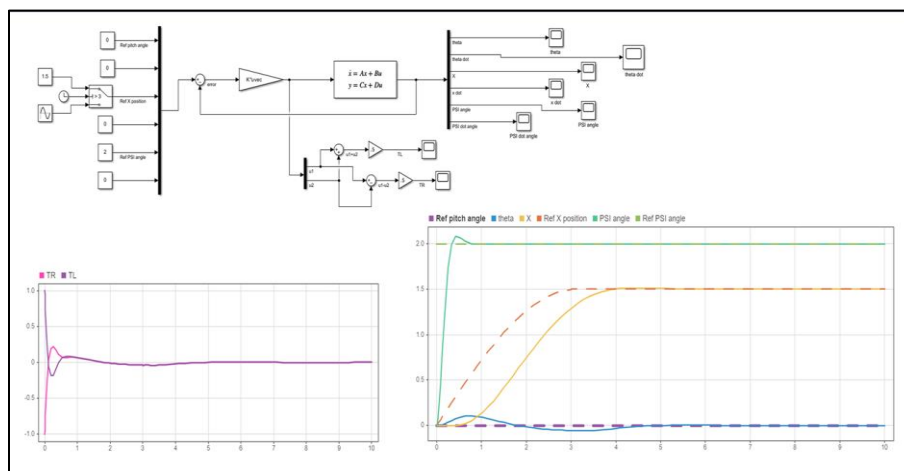


Figure 16 : LQR with 6

8.4 Look-up Polynomial Fitting & LQR Gain Scheduling

A critical limitation in embedded mechatronics is the computational bottleneck of solving complex mathematical models in real-time. Calculating the optimal LQR feedback matrix K requires solving the discrete Algebraic Riccati Equation DARE. Because the A and B matrices in our state-space model are heavily dependent on the time-varying linkage angle (θ_1) the optimal K matrix must also change dynamically. However, continuously solving the DARE on an ESP32 microcontroller at a high control frequency (e.g., 200 Hz) would cause severe loop latency. To bypass this, an offline gain-scheduling architecture was developed.

8.4.1 Offline MATLAB Curve Fitting for Kinematics

To eliminate real-time computational overhead, the non-linear geometric and optimal control calculations were shifted offline. A MATLAB script was developed to sweep through the entire operational range of the upper servo angle $\theta_1 \in [0.5, 1.25]$ rad. For each discrete angular increment, MATLAB evaluated the exact closed-form vector loops to determine the leg length (l_{leg}) and the vertical center of gravity offset (l_w). The script then generated the localized A and B state-space matrices and solved the DARE to extract the optimal K matrix for that specific posture.

Finally, a least-squares curve fitting algorithm was applied to model the relationship between θ_1 and each specific gain value k_i within the K matrix. The resulting approximations took the form of second-order polynomials:

$$k_i(\theta_1) = a_i\theta_1^2 + b_i\theta_1 + c_i$$

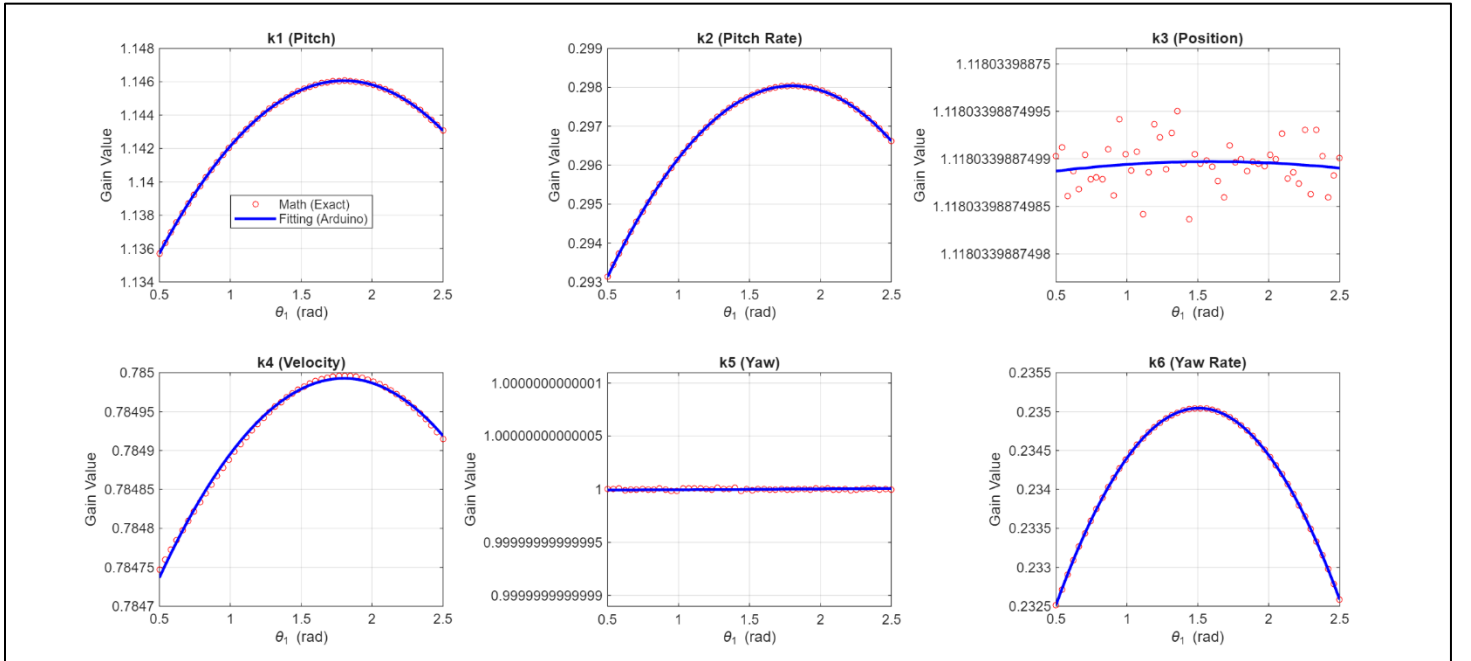


Figure 17 validation k matrix gains

8.4.2 Real-Time LQR Matrix Adjustment

During live operation, the ESP32 executes a highly optimized gain-scheduling routine. By reading the current servo angle θ_1 , the microcontroller evaluates the pre-calculated polynomials to instantly update the K matrix coefficients. This

approach reduces the complex matrix calculus down to a few basic floating-point multiplications and additions per cycle. Consequently, the TWSBR seamlessly scales its control authority, applying stiffer gains when standing tall and softer gains when crouched low—while maintaining an ultra-fast control loop iteration time that prevents instability.

9 Simulation and Experimental Results

9.1 Simscape Multi-Body Simulation Environment

9.1.1 CAD Integration and Virtual Physics Setup

To validate the control algorithms before hardware deployment, a Simscape simulation environment was established. Rather than importing the highly detailed manufacturing CAD, a simplified CAD assembly was utilized to identify the primary components, focusing strictly on accurate mass distribution, inertia tensors, and critical dimensions. A global ground reference was established, and the robot model was linked using appropriate physical joints. This ensured the virtual model reacted physically accurately to the simulated ground and spatial environment, properly respecting the laws of inertia, joint friction, and gravity.

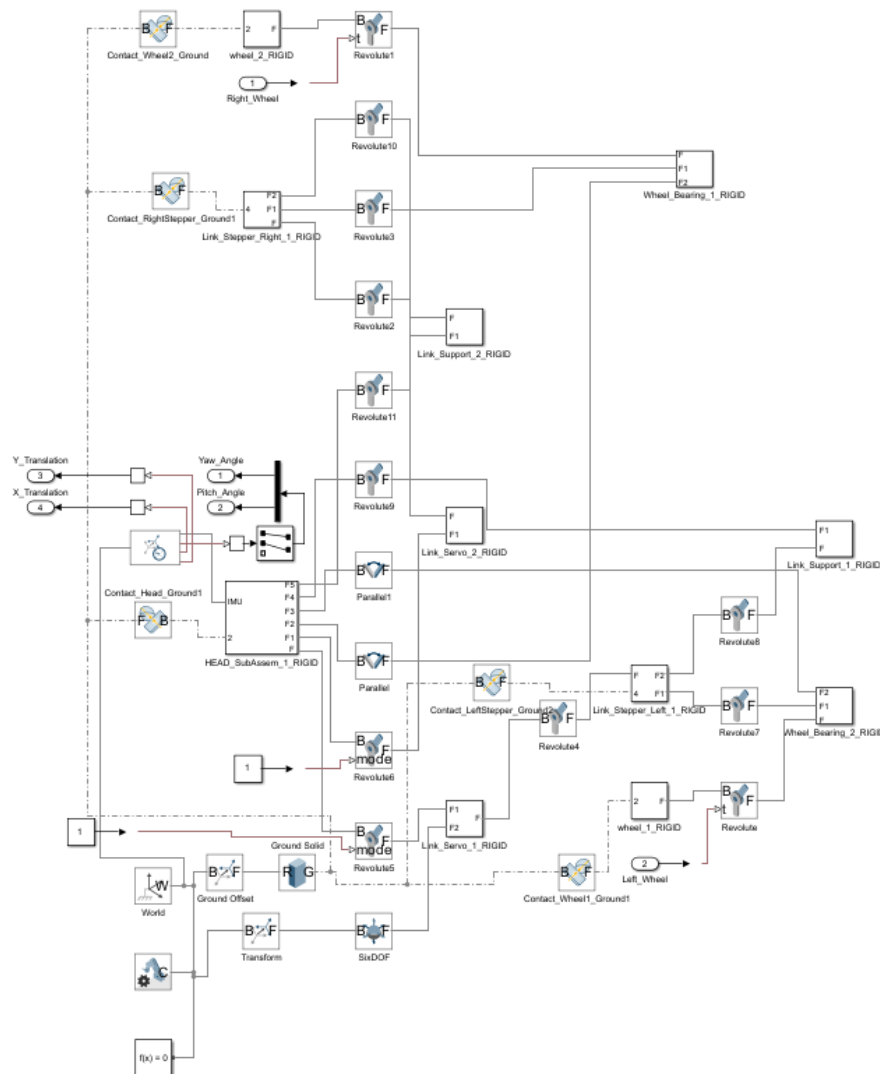


Figure 18 simscape model

9.2 Control Verification

Transitioning directly from mathematical theory to physical hardware carries a high risk of catastrophic failure and hardware damage. To mitigate this, a high-fidelity virtual environment was constructed by importing the SolidWorks CAD assembly into MathWorks Simscape Multibody. This setup captures actual material densities, moments of inertia, and real physical joints, serving as an intermediate stage between ideal block-diagram state-space simulations and physical reality.

Static Pitch Stabilization (Constant θ_1)

The initial simulation phase inside the Simscape physical environment focused entirely on the robot's primary survival mechanism: active self-balancing. For this verification stage, the servo angle θ_1 was held at a constant value, fixing the physical four-bar linkages in place. The control loop was structured around feedback sensing blocks acting as an ideal Inertial Measurement Unit (IMU). In this phase, the LQR tuning process concentrated heavily on the pitch (θ) and pitch rate ($\dot{\theta}$) states to handle balance stabilization. The Simscape model layout integrated sensory feedback blocks, a closed-loop LQR matrix mixer, and physical rotational actuators on the wheel hubs. The simulation confirmed that the derived control framework successfully suppressed gravitational tilt errors, achieving rock-solid upright balance. The collected data plotting curves demonstrated clean convergence back to the zero-pitch vertical axis following external impulse disturbances.

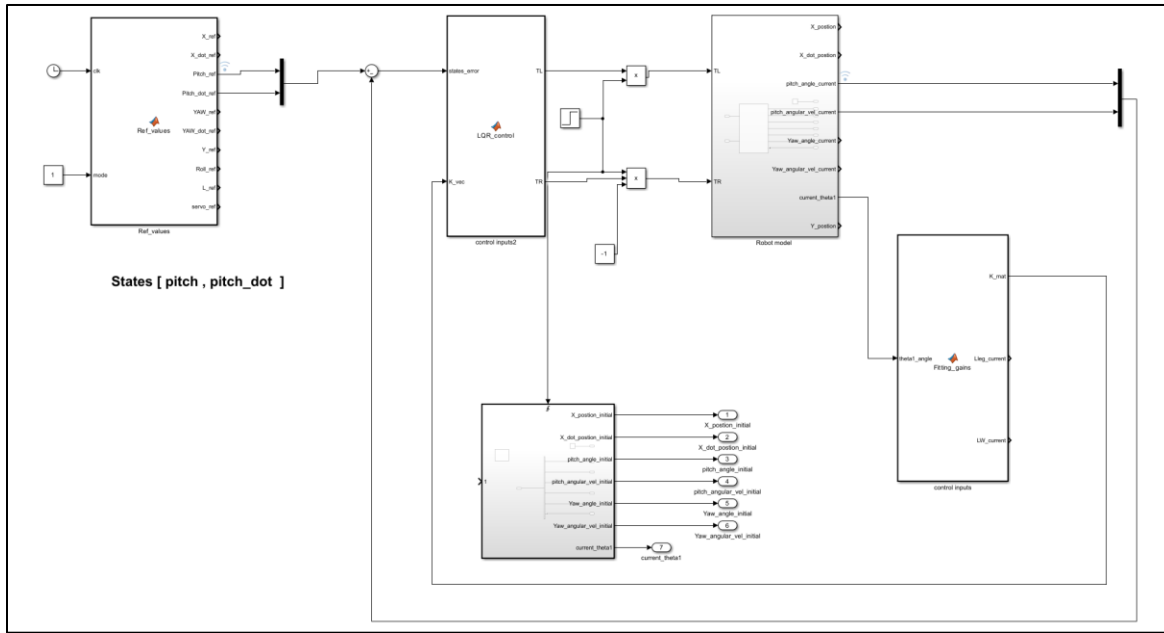


Figure 19 simulink model with simscape multibody

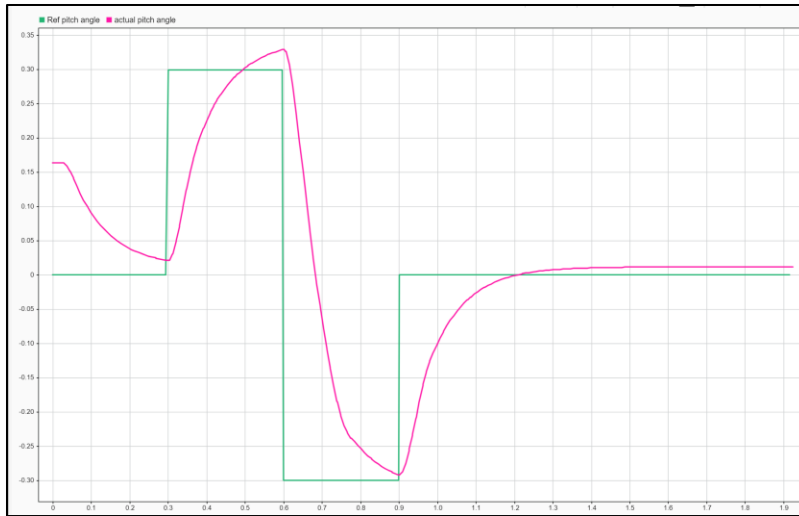


Figure 20 pitch angle vs Ref Pitch with time

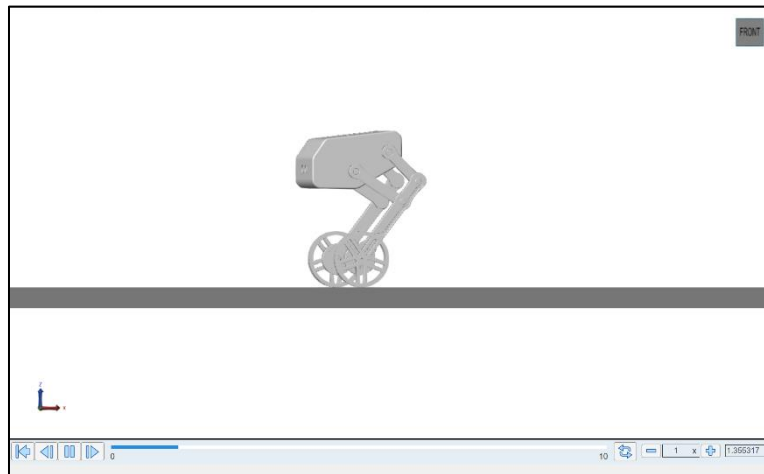


Figure 21 simscape balaning on 0 rad

10 Discussion

The integration of a hybrid control architecture—combining a 4-state and 6-state Linear Quadratic Regulator (LQR) for planar balance and steering tracking alongside a Proportional-Derivative (PD) loop governed by Virtual Model Control (VMC) for height adjustments—successfully met the stability and mobility constraints of the Two-Wheeled Self-Balancing Wheel-Legged Robot (TWSBR). Evaluating these strategies within MathWorks Simulink and Simscape Multibody provided critical engineering insights into the relationship between theoretical design and embedded execution.

10.1 Computational Efficiency of Polynomial Gain Scheduling

A primary hurdle in wheel-legged robotics is the continuous shift in the physical plant behavior; as the four-bar linkage alters the stance height, the system's center of mass (l_w) and rotational inertia (I_p) vary non-linearly. Standard LQR control over variable configurations requires continuously solving the computationally expensive discrete Algebraic Riccati Equation (DARE), an operation that would paralyze the loop frequency of an embedded microcontroller like the ESP32.

The offline MATLAB sweeping pipeline completely bypassed this bottleneck. By curve-fitting the optimal feedback gain matrices (K) into analytical second-order polynomials ($k_i(\theta_1) = a_i\theta_1^2 + b_i\theta_1 + c_i$) mapped directly to the joint angle, the real-time execution constraint was reduced to basic floating-point arithmetic. This approach allowed the ESP32 firmware to maintain an ultra-fast, deterministic control frequency of 200 Hz, preserving control authority and avoiding the loop latencies that induce catastrophic structural oscillation.

10.2 Multi-Environment Simulation and Control Verification

The verification roadmap proved that isolating different control objectives across specialized virtual testing grounds is a highly reliable prototyping pipeline:

1. **High-Fidelity Physical Testing (Simscape Multibody):** Testing inside Simscape focused strictly on the core survival layer: pitch stabilization ($\theta, \dot{\theta}$). Because Simscape maps true 3D CAD material profiles, contact frictions, and multi-body constraints, achieving reliable vertical balancing here provided validation that the mathematical linearization assumptions ($\cos \theta \approx 1, \sin \theta \approx \theta$) were robust against real-world gravity vectors and actuator response latencies.
2. **Numerical State-Space Tracking (Simulink Workspace):** Once pitch balancing was validated under physical constraints, the unlocked 4-state and expanded 6-state models were safely evaluated inside standard Simulink block-diagram workspaces. Testing coordinate target capture and path-following response curves numerically demonstrated that the LQR matrix could multiplex asymmetric wheel *torques* ($T_L \neq T_R$) to negotiate tight turning radii (δ) without injecting disruptive kinetic energy back into the unstable pitch axis.

10.3 Limitations and Future Structural Improvements

While the virtual models achieved high tracking accuracy, several non-ideal physical factors remain unaddressed. The presence of unmodeled backlash in the physical geared actuators and localized wheel slip under rapid yaw accelerations

can degrade pure state feedback tracking over prolonged operations. To eliminate these issues in future hardware iterations, incorporating an active feedforward friction compensation loop alongside a Kalman Filter to blend the wheel encoder data with the MPU6050 IMU telemetry will significantly increase spatial tracking robustness over low-traction operational surfaces.

11 Conclusion

This project successfully designed, modeled, and verified an agile, variable-height Two-Wheeled Self-Balancing Wheel-Legged Robot. By bridging the gap between theoretical multi-body multi-state matrices and embedded hardware limitations, an efficient mechatronic platform was achieved.

The implementation of an offline polynomial gain-scheduling pipeline provided a highly scalable solution to the time-varying dynamics caused by the changing configurations of the four-bar linkage mechanism. Isolating the complex control objectives into specialized testing tiers—evaluating full 3D multi-body pitch stabilization within high-fidelity Simscape environments, alongside full 4-state and 6-state target-following tracking trajectories using Simulink State-Space blocks—provided rigorous mathematical and physical assurance.

The hybrid combination of LQR balancing stability, PD height tracking flexibility, and Virtual Model Control (VMC) joint allocation proved to be an ideal control strategy for wheel-legged architectures. Ultimately, this research provides a lightweight, computationally highly optimized, and structurally responsive framework that establishes a robust foundation for future autonomous navigation and highly dynamic mobile robotic maneuvers.

12 References

- Design and Control of a Wheeled Bipedal Robot Based on Hybrid Linear Quadratic Regulator and Proportional-Derivative Control, School of Mechanical and Automotive Engineering, Shanghai University of Engineering Science, Shanghai, Yanling Tian and Zheng Chen, 2025
- <https://youtu.be/rpD8mo0Jbuc?si=x847OHViFeFNNdov>

13 Appendices

13.1 Appendix A: Low-Level Hardware Diagnostics and Unit Testing Scripts

- A.1: MPU6050 Inertial Measurement Unit (IMU) Telemetry Diagnostics

```
• #include "I2Cdev.h"
• #include "MPU6050_6Axis_MotionApps20.h"
•
• #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
• #include "Wire.h"
• #endif
•
• MPU6050 mpu;
•
• bool dmpReady = false;
• uint8_t devStatus;
• uint16_t packetSize;
• uint8_t fifoBuffer[64];
•
• Quaternion q;
• VectorInt16 gy;
• VectorFloat gravity;
• float ypr[3];
•
• void setup() {
•     Wire.begin();
•     Wire.setClock(400000);
•
•     Serial.begin(115200);
•
•     mpu.initialize();
•
•     Serial.println(mpu.testConnection() ? "MPU6050 Connected" : "MPU6050 Connection
Failed");
•
•     devStatus = mpu.dmpInitialize();
•
•     mpu.setXGyroOffset(220);
•     mpu.setYGyroOffset(76);
•     mpu.setZGyroOffset(-85);
•     mpu.setZAccelOffset(1788);
•
•     if (devStatus == 0) {
•
•         mpu.CalibrateAccel(6);
•         mpu.CalibrateGyro(6);
•
•         mpu.setDMPEnabled(true);
```

```

•     dmpReady = true;
•
•     packetSize = mpu.dmpGetFIFOPacketSize();
•
•     Serial.println("DMP Ready");
• }
• else {
•     Serial.print("DMP Initialization Failed: ");
•     Serial.println(devStatus);
• }
• }
•
• void loop() {
•
•     if (!dmpReady) return;
•
•     if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) {
•
•         mpu.dmpGetQuaternion(&q, fifoBuffer);
•
•         mpu.dmpGetGravity(&gravity, &q);
•
•         mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
•
•         mpu.dmpGetGyro(&gy, fifoBuffer);
•
•         float yaw  = ypr[0] * 180/M_PI;
•         float pitch = ypr[1] * 180/M_PI;
•         float roll  = ypr[2] * 180/M_PI;
•
•         Serial.print("Yaw: ");
•         Serial.print(yaw);
•
•         Serial.print("  Pitch: ");
•         Serial.print(pitch);
•
•         Serial.print("  Roll: ");
•         Serial.print(roll);
•
•         Serial.print("  Gyro Y: ");
•         Serial.println(gy.y / 131.0);
•     }
• }
•

```

- A.2: NEMA 17 Stepper Motor Driver and Microstepping Calibration Test

```

• #define stepPin 3
• #define dirPin 4
• void setup() {

```

```

• pinMode(stepPin, OUTPUT);
• pinMode(dirPin, OUTPUT);
• }
•
• void loop() {
•
•     digitalWrite(dirPin, HIGH);
•
•     for(int x = 0; x < 200; x++) {
•         digitalWrite(stepPin, HIGH);
•         delayMicroseconds(800);
•
•         digitalWrite(stepPin, LOW);
•         delayMicroseconds(800);
•     }
•
•     delay(1000);
•
•     digitalWrite(dirPin, LOW);
•
•     for(int x = 0; x < 200; x++) {
•         digitalWrite(stepPin, HIGH);
•         delayMicroseconds(800);
•
•         digitalWrite(stepPin, LOW);
•         delayMicroseconds(800);
•     }
•
•     delay(1000);
• }

```

13.2 Appendix B: Embedded Control Firmware (C++/Arduino Platform)

- *B.1: Integrated Main Architecture for the Core Mechatronic System*

```

• // I2Cdev and MPU6050 must be installed as libraries
• #include "I2Cdev.h"
• #include "MPU6050_6Axis_MotionApps20.h"
• #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
•     #include "Wire.h"
• #endif
• MPU6050 mpu;
• // MPU control/status vars
• bool dmpReady = false; // set true if DMP init was successful
• uint8_t devStatus;      // return status after each device operation (0 =
                          // success, !=0 = error)
• uint16_t packetSize;    // expected DMP packet size (default is 42 bytes)
• uint8_t fifoBuffer[64]; // FIFO storage buffer
• // orientation/motion vars

```

```

• Quaternion q;           // [w, x, y, z]           quaternion container
• VectorInt16 gy;          // [x, y, z]             gyro sensor measurements (Angular
  Velocity)
• VectorFloat gravity;     // [x, y, z]             gravity vector
• float ypr[3];           // [yaw, pitch, roll]     yaw/pitch/roll container
• ///////////////////////////////////////////////////////////////////
  //
• ///////////////////////////////////////////////////////////////////
  //
• ///////////////////////////////////////////////////////////////////
  //
• ///////////////////////////////////////////////////////////////////
  //
• const float Fixed_Theta1 = 1.0; //Rad
• ///////////////////////////////////////////////////////////////////
  //
• float p_k1[] = {0.074114, -0.173704, -1.929676};
• float p_k2[] = {-0.021690, 0.050877, -0.223459};
• ///////////////////////////////////////////////////////////////////
  //
• float a1 = -0.072848, b1 = 0.205408, c1 = 0.094574;
• float a3 = -0.037861, b3 = 0.088834, c3 = 0.113857;
• ///////////////////////////////////////////////////////////////////
  //
• float Lleg = 0;
• float Lw = 0; // effective Length (pendulum Length)
• float k_pitch, k_pitch_rate;
• ///////////////////////////////////////////////////////////////////
  //
• float current_pitch, current_gyro_y;
• float target_pitch = 0;
• ///////////////////////////////////////////////////////////////////
  //
• ///////////////////////////////////////////////////////////////////
  //
• ///////////////////////////////////////////////////////////////////
  //
• // Left wheel motor
• byte step_L =5;
• byte dir_L =6;
• // Right wheel motor
• byte step_R= 7;
• byte dir_R =8;
•
• long stepDelay = 0;
• unsigned long lastStepTime = 0;
• const int min_delay = 200;
• const int max_delay = 5000;
•
• ///////////////////////////////////////////////////////////////////
  //
• ///////////////////////////////////////////////////////////////////
  //
• ///////////////////////////////////////////////////////////////////
  //
• void setup() {
•   // Arduino Nano uses pins A4 (SDA) and A5 (SCL) automatically
•   Wire.begin();
•   Wire.setClock(400000); // 400kHz I2C clock
•   Serial.begin(115200);
•   while (!Serial);

```

```

• Serial.println(F("Initializing MPU6050..."));
• mpu.initialize();
• // Verify connection
• Serial.println(mpu.testConnection() ? F("MPU6050 connection successful") :
F("MPU6050 connection failed"));
• // Load and configure the DMP
• devStatus = mpu.dmpInitialize();
• // Supply your own gyro offsets here, scaled for min sensitivity
• mpu.setXGyroOffset(220);
• mpu.setYGyroOffset(76);
• mpu.setZGyroOffset(-85);
• mpu.setZAccelOffset(1788);
• if (devStatus == 0) {
•     // Calibration
•     mpu.CalibrateAccel(6);
•     mpu.CalibrateGyro(6);
•
•     Serial.println(F("Enabling DMP..."));
•     mpu.setDMPEnabled(true);
•     dmpReady = true;
•
•     packetSize = mpu.dmpGetFIFOPacketSize();
• } else {
•     Serial.print(F("DMP Initialization failed (code "));
•     Serial.print(devStatus);
•     Serial.println(F(")"));
•
• }
• //////////////////////////////////////////////////Stepper////////////////////////////////////
pinMode(step_L, OUTPUT);
pinMode(dir_L, OUTPUT);
pinMode(step_R, OUTPUT);
pinMode(dir_R, OUTPUT);
•
• //////////////////////////////////////////////////LQR////////////////////////////////////
k_pitch = p_k1[0]*pow(Fixed_Theta1, 2) + p_k1[1]*Fixed_Theta1 + p_k1[2];
k_pitch_rate = p_k2[0]*pow(Fixed_Theta1, 2) + p_k2[1]*Fixed_Theta1 + p_k2[2];
Lleg = a1*pow(Fixed_Theta1, 2) + b1*Fixed_Theta1 + c1;
Lw = a3*pow(Fixed_Theta1, 2) + b3*Fixed_Theta1 + c3;
•
• }
• void loop() {
•     if (!dmpReady) return;
•     // Read a packet from FIFO
•     if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) {
•         // Get orientation data
•         mpu.dmpGetQuaternion(&q, fifoBuffer);
•         mpu.dmpGetGravity(&gravity, &q);
•         mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);

```



```

•     }
•     float abs_torque = abs(torque);
•
•     if (abs_torque < 0.05) {
•         stepDelay = 0;
•     } else {
•
•         stepDelay = 80000 / abs_torque;
•         if (stepDelay < min_delay) stepDelay = min_delay;
•         if (stepDelay > max_delay) stepDelay = max_delay;
•     }
•     run_motors();
• }
•
• void run_motors() {
•     if (stepDelay == 0) return;
•
•     if (micros() - lastStepTime >= stepDelay) {
•         lastStepTime = micros();
•
•         digitalWrite(step_L, HIGH);
•         digitalWrite(step_R, HIGH);
•         delayMicroseconds(10);
•         digitalWrite(step_L, LOW);
•         digitalWrite(step_R, LOW);
•         Serial.println("motor run");
•     }
• }

```

13.3 Appendix C: Dynamic LQR State-Space Controller Implementations

• C.1: 4-State Longitudinal Motion and Pitch Balance Control Script (x-Motion)

```

% --- Physical Constants ---

m = 0.9 ; % wheel mass
M = 2.5 ; % Body Mass
r = 0.05 ; % wheel radius
L = 0.15 ; % length to CoM
Jz = 0.03 ; % body inertia
Iw = 0.000045 ; % wheel inertia
g = 9.81; % gravity

Ew = M + 2*m + (2*Iw/ (r^2)) ;
Ep = (Jz + M*(L^2)) ;
denometer = Ew * Ep - ( (M^2)*(L^2)) ;

A21 = (Ew*M*g*L)/denometer ;
B2 = -1* (Ew + M*L/r) /denometer;
A41 = ( -1* (M^2)*g*(L^2) )/denometer ;
B4 = ( (Ep/r) + M*L )/denometer ;

```



```
A_Mat = [ 0      1      0      0
          A21    0      0      0
          0      0      0      1
          A41    0      0      0 ] ;
```

```
B_Mat = [ 0
          B2
          0
          B4 ] ;
```

```
C_Mat = [ 1  0  0  0
          0  1  0  0
          0  0  1  0
          0  0  0  1 ] ;
```

```
Q = diag([200, 0, 500, 0]);
R = 400 ;
```

```
K = lqr(A_Mat, B_Mat, Q, R) ;
```

- **C.2: 6-State Spatial Trajectory Tracking and Yaw Steering Control Script (X-Y-Motion)**

```
% --- Physical Constants ---
```

```
% XY motion with balancing
```

```
% X state vector = [ 0  0_dot  x  x_dot  δ  δ_dot ]
```

```
% J cost function =  $\int_0^\infty (x^T Q x + u^T R u) dt$ 
```

```
m = 0.9 ; % wheel mass
```

```
M = 2.5 ; % Body Mass
```

```
r = 0.05 ; % wheel radius
```

```
L = 0.15 ; % length to CoM
```

```
Jz = 0.03 ; % body inertia about the yaw axis (horizontal)
```

```
Jy = 0.005 ; % Moment of inertia about the yaw axis (vertical)
```

```
Iw = 0.000045 ; % wheel inertia
```

```
g = 9.81 ; % gravity
```

```
D = 0.15 ; % track width
```

```
Ew = M + 2*m + (2*Iw/ (r^2)) ;
```

```
Ep = (Jz + M*(L^2)) ;
```

```
denometer = Ew * Ep - ( (M^2)*(L^2)) ;
```

```
A21 = (Ew*M*g*L )/denometer ;
```

```
B2 = -1* (Ew + M*L/r) /denometer;
```

```
A41 = ( -1* (M^2)*g*(L^2) )/denometer ;
```

```
B4 = ( (Ep/r) + M*L )/denometer ;
```

```
B6 = 1 /(r*(m*D + (Iw*D)/(r^2) + 2*Jy/D)) ;
```

```
A_Mat = [ 0      1      0      0      0      0
          A21    0      0      0      0      0
          0      0      0      1      0      0
          A41    0      0      0      0      0
          0      0      0      0      0      1
          0      0      0      0      0      0 ] ;
```

```

B_Mat = [ 0      0
          B2     0
          0      0
          B4     0
          0      0
          0      B6] ;

C_Mat = [ 1  0      0  0  0  0
          0  1      0  0  0  0
          0  0      1  0  0  0
          0  0      0  1  0  0
          0  0      0  0  1  0
          0  0      0  0  0  1 ] ;

D_Mat = [ 0      0
          0      0
          0      0
          0      0
          0      0
          0      0] ;

Q = diag([200, 0, 500, 0 , 300 , 0 ]);
R = diag([400 , 300 ]) ;

K = lqr(A_Mat, B_Mat, Q, R) ;

```

13.4 Appendix D: High-Level Simulation and Control Initialization Scripts

• D.1: Simulink LQR State-Space Block Initialization and Workspace Configuration Script

```

% --- Physical Constants ---

m = 0.1 ; % wheel mass
M = 0.9 ; % Body Mass
r = 0.05 ; % wheel radius
L = 0.15 ; % length to CoM
Jz = 0.03 ; % body inertia
Iw = 0.000045 ; % wheel inertia
g = 9.81; % gravity

Ew = M + 2*m + (2*Iw/ (r^2)) ;
Ep = (Jz + M*(L^2)) ;
denometer = Ew * Ep - ( (M^2)*(L^2)) ;

A21 = (Ew*M*g*L )/denometer ;
B2 = -1* (Ew + M*L/r) /denometer;
A41 = ( -1* (M^2)*g*(L^2) )/denometer ;
B4 = ( (Ep/r) + M*L )/denometer ;

A_Mat = [ 0      1      0      0
          A21     0      0      0
          0      0      0      1
          A41     0      0      0 ] ;

B_Mat = [ 0
          B2
          0

```

```

        B4 ] ;

C_Mat = [ 1  0  0  0
          0  1  0  0
          0  0  1  0
          0  0  0  1 ] ;

Q = diag([200, 0, 500, 0]);
R = 400 ;

K = lqr(A_Mat, B_Mat, Q, R) ;

```

- **D.2: Inverse Kinematics Function Block**

```

function [theta1_cmd, theta6_cmd] = fcn(l_desired, l1, l2, l3, l4, l23)
% Compute inverse mapping configuration for upper actuator input
arg1 = (l_desired^2 + l2^2 - l1^2) / (2 * l_desired * l2);
arg1 = max(min(arg1, 1), -1); % Bound argument to prevent imaginary roots
theta1_cmd = asin(arg1);

% Resolve intermediate auxiliary vector distance
l5 = sqrt(l2^2 + l4^2 - 2*l2*l4*cos(theta1_cmd + pi/4));

% Evaluate downstream inter-link angle transformations
theta2 = asin((l4 * sin(theta1_cmd + pi/4)) / l5);

arg3 = (l23^2 + l5^2 - l3^2) / (2 * l23 * l5);
arg3 = max(min(arg3, 1), -1);
theta3 = acos(arg3);

theta4 = pi - theta1_cmd - theta2 - theta3;

% Formulate lower link companion positioning command
arg6 = (l5 * sin(theta3)) / l3;
arg6 = max(min(arg6, 1), -1);
theta6_cmd = asin(arg6) - theta4;
end

```

D.3:: Closed-Form Forward Leg Kinematics and COM Profiler (Leg_Kinematics.m)

```

function [lleg, L0, lw] = fcn(theta1, l1, l2, l3, l4, l23, m1, m2, m3, m23)
% Reconstruct dynamic loop-closure parameters from instantaneous state
l5 = sqrt(l2^2 + l4^2 - 2 * l2 * l4 * cos(theta1 + pi/4));

arg_asin = (l4 * sin(theta1 + pi/4)) / l5;
arg_asin = max(min(arg_asin, 1), -1);
theta2 = asin(arg_asin);

arg_acos = (l23^2 + l5^2 - l3^2) / (2 * l23 * l5);
arg_acos = max(min(arg_acos, 1), -1);
theta3 = acos(arg_acos);

theta4 = pi - theta1 - theta2 - theta3;

```

```

theta5 = pi - theta4;

arg_asin_theta6 = (l5 * sin(theta3)) / l3;
arg_asin_theta6 = max(min(arg_asin_theta6, 1), -1);
theta6 = asin(arg_asin_theta6) - theta4;

% Calculate exact output linear height profile
lleg = l2 * sin(theta1) + l1 * sin(theta4);

% Determine discrete spatial mass center coordinates for individual components
xc2 = (l2 * sin(theta1)) / 2;
xc1 = l2 * sin(theta1) + (l1 * sin(theta5)) / 2;
xc3 = -l4 * sin(pi/4) + (l3 * sin(theta6)) / 2;
xc23 = l2 * sin(theta1) - (l23 * sin(theta4)) / 2;

% Mass-weighted composite center of gravity tracking calculation
L0 = (m1*xc1 + m2*xc2 + m3*xc3 + m23*xc23) / (m1 + m2 + m3 + m23);

% Global vertical system center of mass displacement offset
lw = lleg - L0;
end

```

D.4: Virtual Model Control (VMC) Jacobian Mapping Function (Virtual_Model_Control.m)

```

function [T_link2, l5] = fcn(F0, theta1, l1, l2, l3, l4, l23)
% Compute instantaneous linkage frame spatial transformation vectors
l5 = sqrt(l2^2 + l4^2 - 2*l2*l4*cos(theta1 + pi/4));

alpha_val = asin((l4 * sin(theta1 + pi/4)) / l5);

cos_beta_arg = (l23^2 + l5^2 - l3^2) / (2 * l23 * l5);
cos_beta_arg = max(min(cos_beta_arg, 1), -1);
beta_val = acos(cos_beta_arg);

% Formulate scalar component configuration entry for the System Jacobian
J1 = l2 * cos(theta1) + l1 * cos(theta1 + alpha_val + beta_val);

% Translate operational structural load to joint-space motor output torque
T_link2 = F0 * J1;
end

```

D.5: Compliance Actuator Compliance and Deflection Mapping (Servo_Actuator.m)

```

function theta_final = fcn(theta_cmd, torque_vmc, joint_stiffness)
% Compute physical angular link displacement caused by loading profile
delta_theta = torque_vmc / joint_stiffness;

% Superimpose elastic structural deflection onto nominal angle positioning command
theta_final = theta_cmd + delta_theta;
end

```

13.5 Appendix E: Offline MATLAB Numerical Fitting and Matrix Scheduling Scripts

- E.1: Least-Squares Kinematic Geometry Fitting (l_{leg} and l_w Profile Approximation vs. θ_1)

```

% =====
% TWSB Robot: Kinematic Fitting & Validation Script
% Added: Comparison plots for Leg Length and CG Offset accuracy
% =====
clear; clc; close all;

% --- 1. Physical Constants ---
l1 = 0.5; l2 = 0.15; l3 = 0.15; l4 = 0.1; l23 = 0.1;
m1 = 0.2; m2 = 0.3; m3 = 0.1; m23 = 0.8;
m_legs = m1 + m2 + m3 + m23;

% --- 2. Data Generation Loop ---
theta1_range = linspace(0.5, 1.25, 100);
Lleg_data = zeros(size(theta1_range));
LW_data = zeros(size(theta1_range));

for i = 1:length(theta1_range)
    theta1 = theta1_range(i);

    % Kinematic equations
    l5 = sqrt(l2^2 + l4^2 - 2 * l2 * l4 * cos(theta1 + pi/4));
    theta2 = asin(max(-1, min(1, (l4 * sin(theta1 + pi/4)) / l5)));
    theta3 = acos(max(-1, min(1, (l23^2 + l5^2 - l3^2) / (2 * l23 * l5))));
    theta4 = pi - theta1 - theta2 - theta3;
    theta5 = pi - theta4;
    theta6 = asin(max(-1, min(1, (l5 * sin(theta3)) / l3))) - theta4;

    lleg = l2 * sin(theta1) + l1 * sin(theta4);

    xc2 = (l2 * sin(theta1)) / 2;
    xc1 = l2 * sin(theta1) + (l1 * sin(theta5)) / 2;
    xc3 = -l4 * sin(pi/4) + (l3 * sin(theta6)) / 2;
    xc23 = l2 * sin(theta1) - (l23 * sin(theta4)) / 2;

    L0 = (m1*xc1 + m2*xc2 + m3*xc3 + m23*xc23) / m_legs;
    lw = lleg - L0;

    Lleg_data(i) = lleg;
    LW_data(i) = lw;
end

% --- 3. Perform Quadratic Fittings ---
p_theta1_lleg = polyfit(theta1_range, Lleg_data, 2);
p_theta1_lw = polyfit(theta1_range, LW_data, 2);

% Calculate fitted values for validation
Lleg_fit = polyval(p_theta1_lleg, theta1_range);
Lw_fit = polyval(p_theta1_lw, theta1_range);

% --- 4. Original Visualization (Fitting Curves) ---
figure('Name', 'Fitting Analysis', 'Color', 'w', 'Position', [50, 400, 1200, 400]);
subplot(1, 3, 1);
plot(theta1_range, Lleg_data, 'ro', 'MarkerSize', 3); hold on;
plot(theta1_range, Lleg_fit, 'b-', 'LineWidth', 2);
grid on; title('\theta_1 \rightarrow l_{leg}'); ylabel('m');

subplot(1, 3, 2);
plot(Lleg_data, LW_data, 'go', 'MarkerSize', 3); hold on;
[lleg_sorted, idx] = sort(Lleg_data);
p_lleg_lw = polyfit(Lleg_data, LW_data, 2);

```

```

plot(lleg_sorted, polyval(p_lleg_lw, lleg_sorted), 'b-', 'LineWidth', 2);
grid on; title('L_{leg} \rightarrow L_w');

subplot(1, 3, 3);
plot(theta1_range, LW_data, 'mo', 'MarkerSize', 3); hold on;
plot(theta1_range, Lw_fit, 'b-', 'LineWidth', 2);
grid on; title('\theta_1 \rightarrow L_w');

% --- 5. Validation Window (Real vs Fit Comparison) ---
figure('Name', 'Accuracy Validation', 'Color', 'w', 'Position', [50, 50, 1000, 500]);

% Subplot A: Leg Length Comparison
subplot(2, 2, 1);
plot(theta1_range, Lleg_data, 'k', 'LineWidth', 2); hold on;
plot(theta1_range, Lleg_fit, 'r--', 'LineWidth', 1.5);
grid on; legend('Real (Eq)', 'Fitted');
ylabel('L_{leg} (m)'); title('Leg Length: Real vs Quadratic Fit');

% Subplot B: Leg Length Error
subplot(2, 2, 3);
plot(theta1_range, abs(Lleg_data - Lleg_fit), 'd-');
grid on; ylabel('Abs Error (m)'); xlabel('Servo Angle (rad)');
title('L_{leg} Fitting Residuals');

% Subplot C: Lw Comparison
subplot(2, 2, 2);
plot(theta1_range, LW_data, 'k', 'LineWidth', 2); hold on;
plot(theta1_range, Lw_fit, 'm--', 'LineWidth', 1.5);
grid on; legend('Real (Eq)', 'Fitted');
ylabel('L_w (m)'); title('CG Offset (Lw): Real vs Quadratic Fit');

% Subplot D: Lw Error
subplot(2, 2, 4);
plot(theta1_range, abs(LW_data - Lw_fit), 'd-', 'Color', [0.5 0 0.5]);
grid on; ylabel('Abs Error (m)'); xlabel('Servo Angle (rad)');
title('L_w Fitting Residuals');

% --- 6. Output Coefficients ---
fprintf('\n--- COPY THESE COEFFICIENTS TO YOUR ARDUINO CODE ---\n');
fprintf('// Fitting: theta1 -> Lleg\n');
fprintf('float a1 = %f; float b1 = %f; float c1 = %f;\n', p_theta1_lleg);
fprintf('\n// Fitting: Lleg -> Lw\n');
fprintf('float a2 = %f; float b2 = %f; float c2 = %f;\n', p_lleg_lw);
fprintf('\n// Fitting: theta1 -> Lw\n');
fprintf('float a3 = %f; float b3 = %f; float c3 = %f;\n', p_theta1_lw);

```

- E.2: Optimal Feedback Gain Curve-Fitting and Polynomial lookup Generation (KMatrix Allocation vs. θ_1)

```

% =====
% LQR Validation Script: Exact Gains vs. Polynomial Fitting
% =====

clear; clc; close all;

% --- 1. System Constants ---
theta1_range = linspace(0.5, 2.5, 50);
m_w = 0.2; mb = 0.9; ml = 0.4; mp = mb + 2*ml;
r = 0.05; g = 9.81; D = 0.15; lb = 0.12;

```

```

Jz_body = 0.033; Iw = 0.00025;
Jy_base = 0.008;

% Geometry fittings (Inputs)
a_lw = -0.010700; b_lw = 0.038542; c_lw = 0.345832;
a_lleg = -0.087522; b_lleg = 0.263838; c_lleg = 0.307096;

% LQR Weights
Q = diag([200, 10, 500, 10, 300, 10]);
R = diag([400, 300]);

% --- 2. Step-by-Step Mathematical Calculation (Exact Data) ---
K_exact = zeros(length(theta1_range), 6); % To store 6 gains for each angle

for i = 1:length(theta1_range)
    t1 = theta1_range(i);

    % Update physics exactly for this step
    lw = a_lw*t1^2 + b_lw*t1 + c_lw;
    lleg = a_lleg*t1^2 + b_lleg*t1 + c_lleg;
    L = (lb*mb + lw*(2*m1)) / mp;
    Ip = Jz_body + mb*(lb - L)^2 + (2*m1)*(L - lw)^2;
    Jy = Jy_base + (2 * m1 * (D/2)^2) * (lleg / 0.45);

    % Matrix construction
    Ew = mp + 2*m_w + (2*Iw / (r^2));
    Ep = Ip;
    denom = Ew * Ep - (mp^2 * L^2);

    A21 = (Ew * mp * g * L) / denom;
    B2 = -1 * (Ew + mp * L / r) / denom;
    A41 = (-1 * mp^2 * g * L^2) / denom;
    B4 = (Ep / r + mp * L) / denom;
    B6 = 1 / (r * (m_w * D + (Iw * D) / (r^2) + 2 * Jy / D));

    A = [0 1 0 0 0 0; A21 0 0 0 0 0; 0 0 0 1 0 0; A41 0 0 0 0 0; 0 0 0 0 0 1; 0 0 0 0 0 0];
    B = [0 0; B2 0; 0 0; B4 0; 0 0; 0 B6];

    % The Mathematical Truth
    K_temp = lqr(A, B, Q, R);
    K_exact(i, :) = [K_temp(1,1:4), K_temp(2,5:6)]; % Store relevant gains
end

% --- 3. Create Polynomial Fittings ---
titles = {'k1 (Pitch)', 'k2 (Pitch Rate)', 'k3 (Position)', 'k4 (Velocity)', 'k5 (Yaw)', 'k6 (Yaw Rate)'};
p_coeffs = cell(1,6);

figure('Color', 'w', 'Name', 'LQR Fitting Validation');

for j = 1:6
    % Fit the polynomial to the exact data
    p_coeffs{j} = polyfit(theta1_range, K_exact(:,j), 2);

    % Generate fitted line for plotting
    K_fit = polyval(p_coeffs{j}, theta1_range);

    % Plotting
    subplot(2, 3, j);

```

```
plot(theta1_range, K_exact(:,j), 'ro', 'MarkerSize', 4, 'DisplayName', 'Math  
(Exact)'); hold on;  
plot(theta1_range, K_fit, 'b-', 'LineWidth', 2, 'DisplayName', 'Fitting (Arduino)');  
grid on; title(titles{j});  
xlabel('\theta_1 (rad)'); ylabel('Gain Value');  
if j == 1, legend('Location', 'best'); end  
end
```